Terraform and AWS Infrastructure for Twenty CRM

1. Objective

Provision AWS infrastructure consisting of an EC2 instance and an Amazon ECR repository using Terraform, while reusing the account's existing default VPC. No new VPC is created. The Twenty CRM Docker image is pushed to ECR, and the EC2 instance automatically authenticates to ECR, pulls the image, and runs the application through a boot-time user-data script with retry logic.

2. File-by-File Explanation

provider.tf

Declares the required Terraform version (>= 1.5.0) and AWS provider (~> 5.0), pinned to us-east-1. The configuration uses local Terraform state because this is a single-person, short-lived deployment. default_tags automatically applies Project, ManagedBy, and Environment tags to resources.

variables.tf

Centralizes every configurable deployment value:

- Networking: use_default_vpc, existing_vpc_id, existing_subnet_id. These allow the configuration to auto-discover the default VPC/subnet or target a specific one.
- EC2: instance_type, key_pair_name, ssh_ingress_cidr, app_port, root_volume_size, ami_id.
- ECR: ecr_repository_name, ecr_image_tag, image_mutability.
- IAM: iam_instance_profile_name. This references the pre-existing EC2ECRPullRole instance profile rather than creating a new IAM role because the AWS IAM user lacks iam:CreateRole permission.

main.tf

- Networking data sources: data.aws_vpcs (plural) is used to look up the default VPC instead of data.aws_vpc (singular). The singular data source also calls ec2:DescribeVpcAttribute for DNS settings, which this IAM user is not authorized to perform. data.aws_subnets then finds subnets inside the selected VPC. No aws_vpc resource is created.
- AMI lookup: data.aws_ami dynamically retrieves the latest Canonical Ubuntu 20.04 AMI instead of relying on a hardcoded AMI ID that could become outdated.
- ECR repository: aws_ecr_repository.twenty_crm creates a private repository with scan_on_push = true for basic vulnerability scanning and force_delete = true so terraform destroy can remove the repository after an image has been pushed.
- Security group: aws_security_group.twenty_crm_sg allows SSH on port 22 only from var.ssh_ingress_cidr and exposes the application port 2020 to the internet so the CRM UI can be reached. Outbound traffic is open.
- EC2 instance: aws_instance.twenty_crm launches into the discovered default subnet, attaches the security group and pre-existing IAM instance profile, and renders templates/user_data.sh.tpl with the ECR repository URL, image tag, AWS region, and application port.
- user_data_replace_on_change = true causes the instance to be recreated when the user-data script changes. This mechanism was used when the application port was corrected from 3000 to 2020.

templates/user_data.sh.tpl

The script runs automatically when the EC2 instance boots:

1. Installs Docker and starts the Docker service.
2. Authenticates to Amazon ECR using aws ecr get-login-password and the EC2 instance's IAM instance profile. No AWS credentials are hardcoded in the script.
3. Loops and retries docker pull <ecr_repo_url>:<tag> every 30 seconds until the image becomes available. This allows the instance to boot before the image is pushed and recover automatically.
4. Runs the application container with docker run -d --name twenty-crm -p 2020:2020 --restart unless-stopped <ecr_repo_url>:<tag>.

outputs.tf

Exports the information needed to interact with the deployment, including the ECR repository URL and ARN, EC2 instance ID, public IP, public DNS name, computed application URL, and the VPC and subnet IDs used.

terraform.tfvars

Stores the actual deployment values for this run, including project_name = "twenty-crm-fathima", app_port = 2020, the personal EC2 key pair name, and the personal IP address used for SSH ingress. This file is gitignored and should not be committed to the repository.

3. Step-by-Step Deployment Process

Step 1 — Provision Infrastructure

The following Terraform commands were executed:

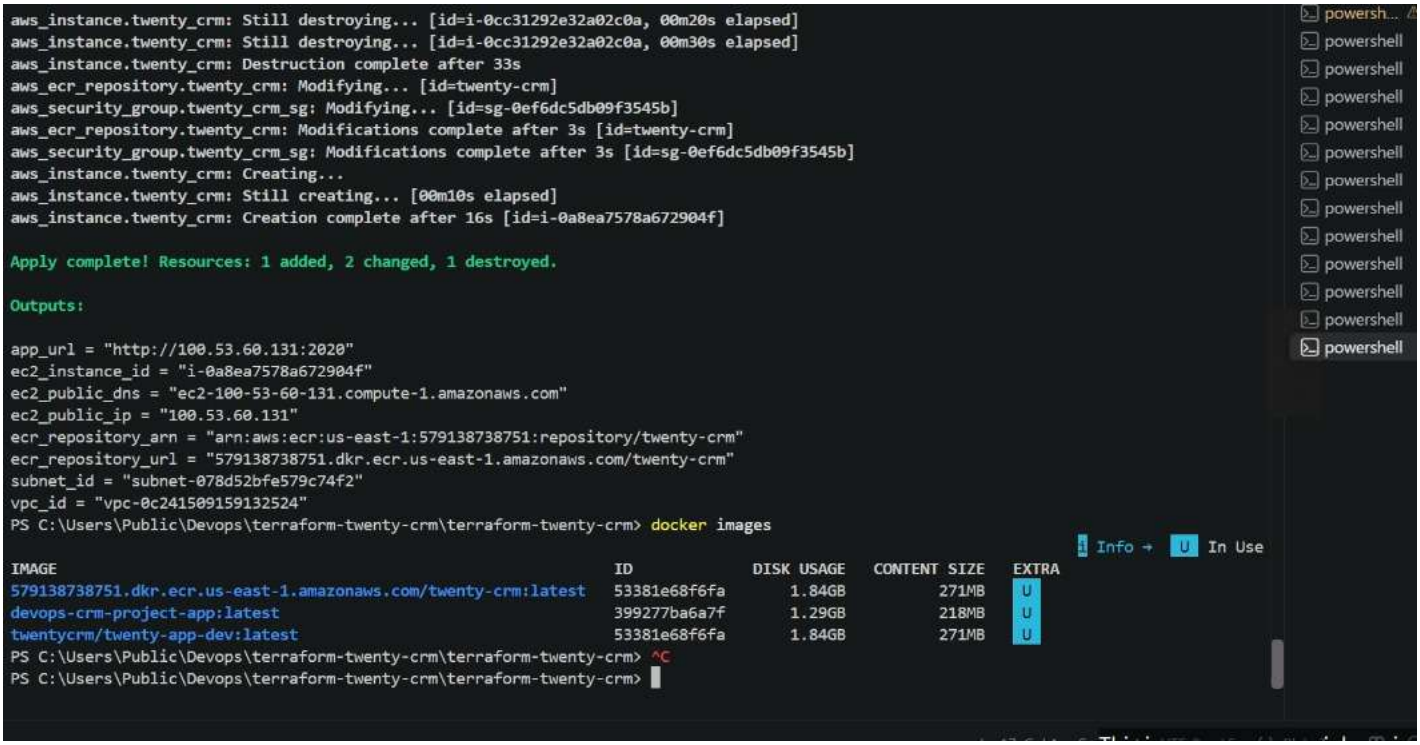
```
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

Final Terraform apply result:

```
Apply complete! Resources: 1 added, 2 changed, 1 destroyed.

Outputs:
app_url             = "http://100.53.60.131:2020"
ec2_instance_id     = "i-0a8ea7578a672904f"
ec2_public_dns      = "ec2-100-53-60-131.compute-1.amazonaws.com"
ec2_public_ip       = "100.53.60.131"
ecr_repository_arn  = "arn:aws:ecr:us-east-1:579138738751:repository/twenty-crm"
ecr_repository_url  = "579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm"
subnet_id           = "subnet-078d52bfe579c74f2"
vpc_id              = "vpc-0c241509159132524"
```

The one destroyed resource reflects the earlier EC2 instance being replaced after the user-data script was corrected. The replacement ensured the updated application port and boot-time configuration were applied.



Step 2 — Identify the Correct Docker Image

Two local Docker images were available:

- devops-crm-project-app:latest — a custom build from the repository's Dockerfile. It is a development/synchronization helper image, not the actual CRM application.
- twentycrm/twenty-app-dev:latest — the official all-in-one Twenty CRM image. It bundles the server and its own PostgreSQL database and serves the application on port 2020.

The second image was confirmed as the correct image to deploy.

Step 3 — Tag and Push the Image to ECR

The Twenty CRM image was tagged with the ECR repository URL:

```
docker tag twentycrm/twenty-app-dev:latest 579138738751.dkr.ecr.us-east-
1.amazonaws.com/twenty-crm:latest
```

ECR authentication was performed with:

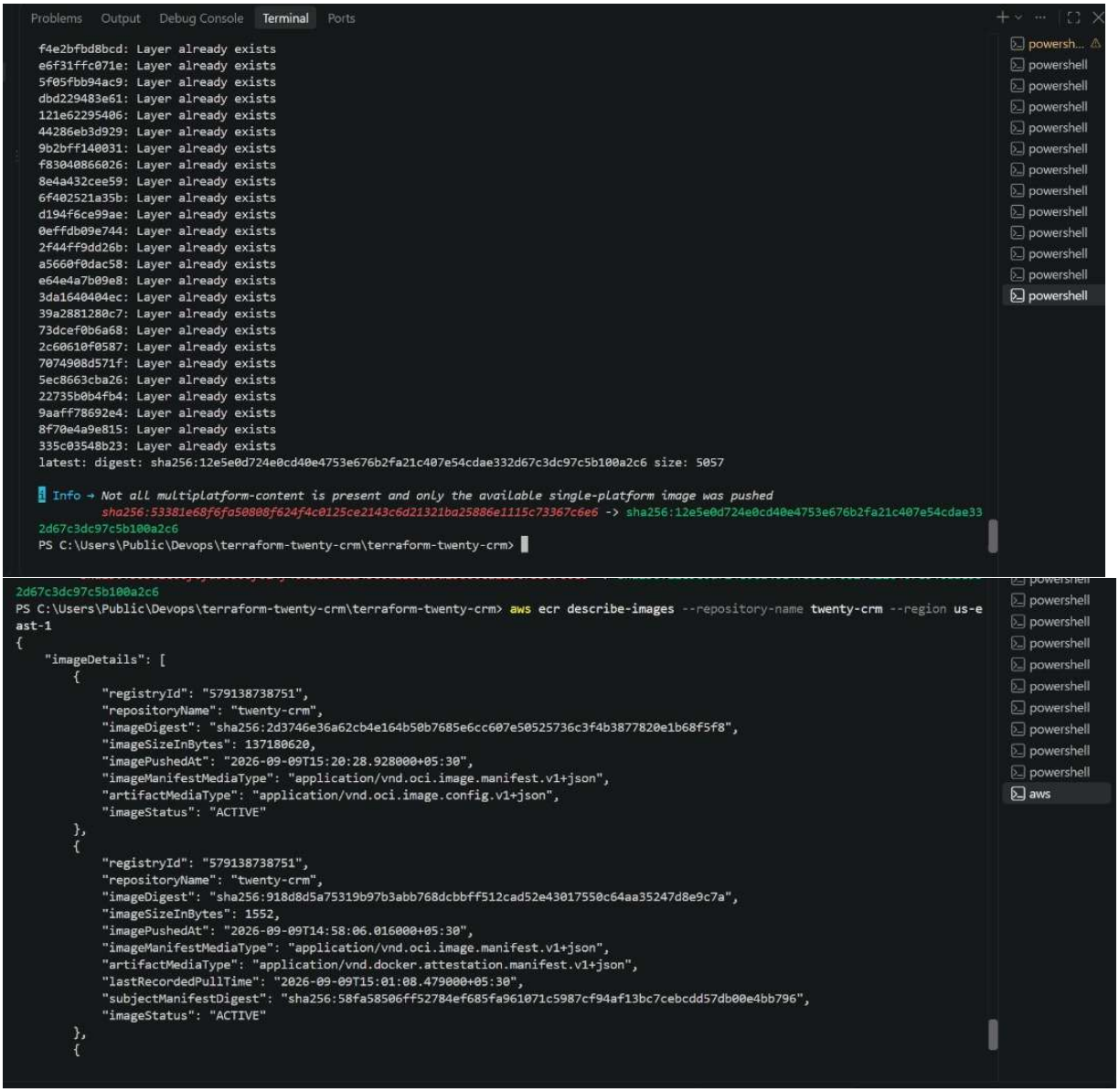
```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
579138738751.dkr.ecr.us-east-1.amazonaws.com
```

The image was then pushed:

```
docker push 579138738751.dkr.ecr.us-east-1.amazonaws.com/twenty-crm:latest
```

The local tag was verified to point to the same image ID as the source image, 53381e68f6fa, confirming a clean retag rather than a rebuild. The push was also verified in ECR with:

```
aws ecr describe-images --repository-name twenty-crm --region us-east-1
```

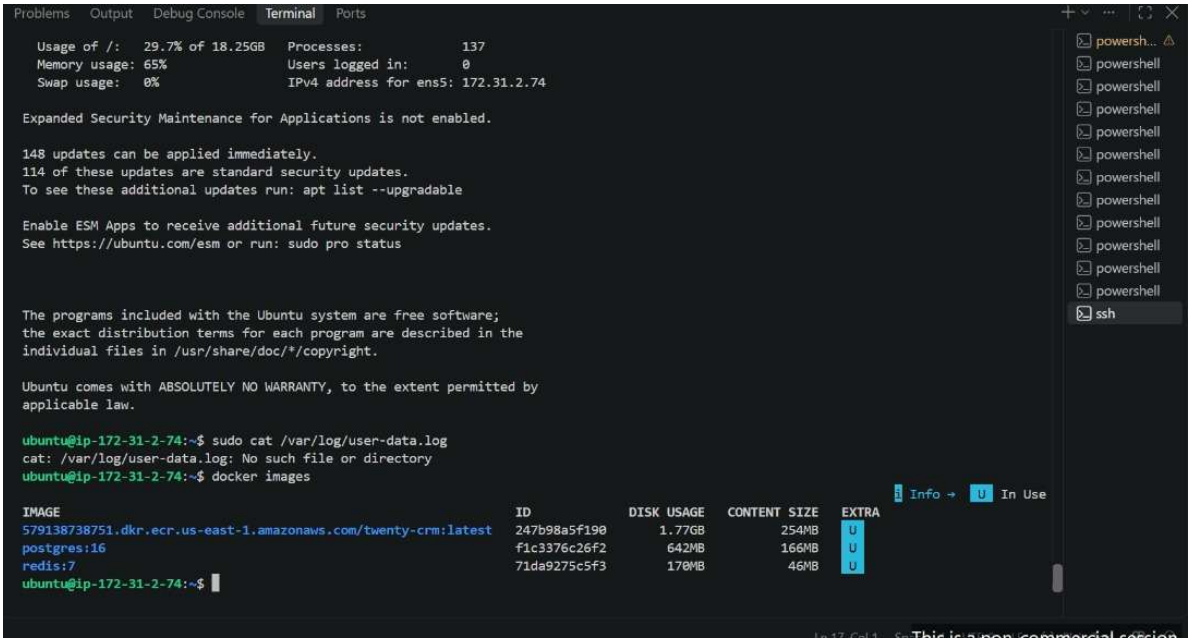


Step 4 — EC2 Boot Automation

During boot, the EC2 user-data script installed Docker, authenticated to ECR using the attached IAM instance profile, and entered the retry loop. Once the image became available in ECR, the script pulled it automatically and started the Twenty CRM container without manual intervention.

The deployment was checked using:

```
ssh -i twenty-crm-key.pem ubuntu@100.53.60.131
docker images
```



Step 5 — Verify the Application

The application endpoint was tested with:

```
curl http://100.53.60.131:2020
```

The application was then opened in a browser at http://100.53.60.131:2020. The Twenty CRM UI loaded successfully, confirming that the application was running end-to-end on EC2 and that the image had been pulled from the private ECR repository.

4. Cleanup

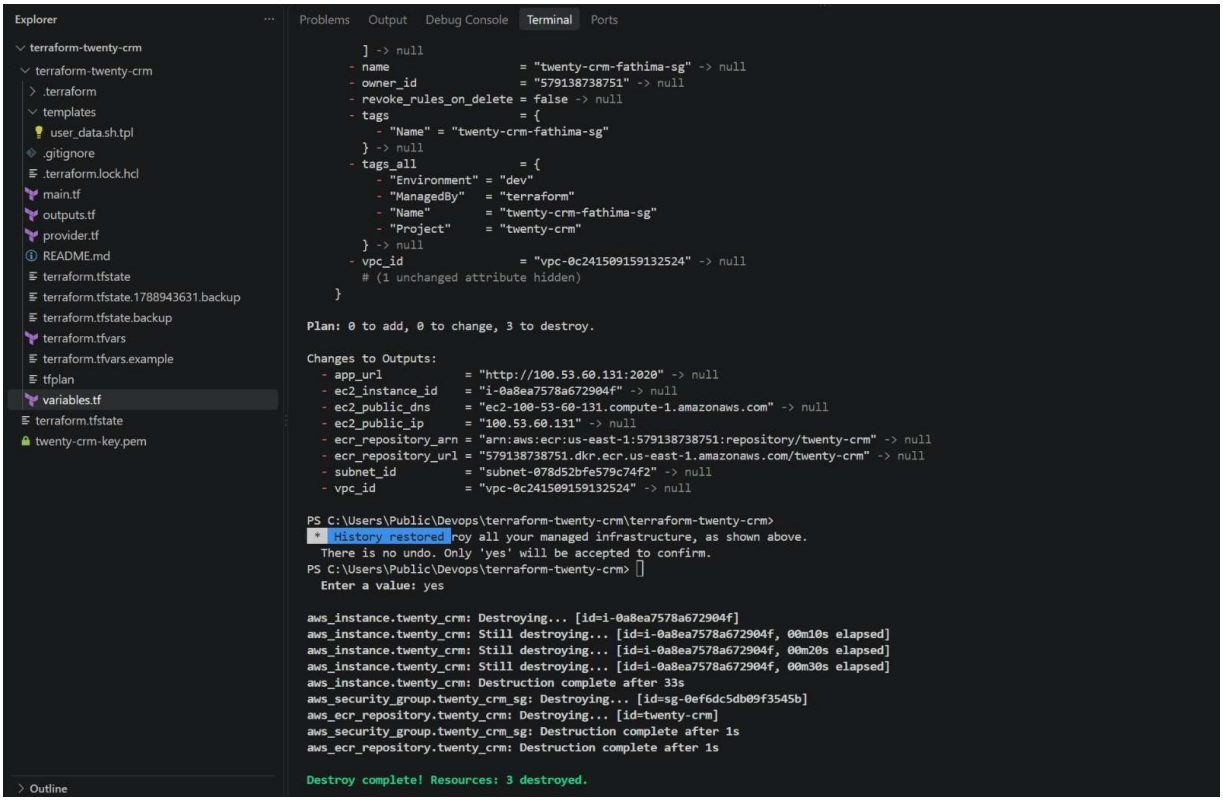
After verification, all Terraform-managed resources were removed using:

```
terraform destroy
```

Cleanup was confirmed with the following AWS CLI commands:

```
aws ec2 describe-instances --instance-ids i-0a8ea7578a672904f --region us-east-1
aws ecr describe-repositories --repository-names twenty-crm --region us-east-1
```

Both commands returned "not found", confirming that the EC2 instance was terminated and the ECR repository, including its pushed image, was removed successfully.



5. Conclusion

Terraform successfully provisioned the required ECR repository and EC2 instance in the account's existing default VPC without creating a new VPC. The Twenty CRM Docker image was tagged, pushed to the private ECR repository, and automatically pulled and started on EC2 through a self-healing user-data script with retry logic. The application was verified through the browser at http://100.53.60.131:2020, and the deployment was fully cleaned up afterward with AWS CLI verification.